Anomaly Detection Exercise Set

Z-score • Modified Z-score (MAD) • KNN

Dataset: `daily_sales.csv`, `product_reference.csv`

---

0. Overview

You have two years (2023-01-01 to 2024-12-31) of daily unit sales for four products (`P001` – `P004`). The data contains genuine demand patterns (trend, weekly/yearly seasonality, and for P003, deliberately lumpy/intermittent demand) plus a number of injected anomalies: point spikes, stockout zero-runs, sustained level shifts, data-entry errors, missing rows, duplicate rows, and one subtle gradual drift.

This exercise set covers exactly three detection methods, in increasing order of sophistication:

1. **Z-score** — the simplest statistical outlier test.
2. **Modified Z-score (MAD-based)** — a robust alternative that resists being thrown off by the very outliers it's trying to find.
3. **KNN (K-Nearest Neighbors) distance-based detection** — a method that considers a point's *neighborhood* rather than a single global distribution, and can work on multiple features at once instead of just raw sales.

Files

File	Description
`daily_sales.csv`	Columns: `product_id`, `date`, `sales`. Note this file is **not** a perfect grid — some product/date rows are missing, and a few appear twice.
`product_reference.csv`	Plain-English description of each product's expected pattern. Read this before starting.

Learning objectives

By the end you should be able to:

- Compute and apply Z-score and modified Z-score thresholds correctly **per product**.
- Explain *why* modified Z-score is more robust than Z-score, and demonstrate it numerically rather than just stating it.
- Build a feature space and apply KNN distance-based anomaly detection, understanding how it differs from a purely univariate threshold test.

Tools

Python (pandas, numpy, scikit-learn for KNN) is assumed. SQL equivalents are noted where relevant for Z-score and modified Z-score, since those are simple aggregate computations; KNN is Python-only for this exercise set.

Exercise 1 — Z-score Detection

Goal: Learn the simplest outlier test and its failure modes.

1. Load `daily_sales.csv`. For **each product independently**, compute the mean and standard deviation of `sales`, and the Z-score of every observation: `z = (x - mean) / std`.
2. Flag any row with `|z| > 3` as an anomaly candidate. Report how many candidates were flagged per product.

   

3. **Diagnostic question — global vs. local baseline:** P002 has a sustained level shift injected partway through the series (a permanent step-up in typical sales). Recompute the mean/std for P002 separately for the period *before* the shift and the period *after*, and compare both to the single mean/std you computed over the full two years. Explain what this does to your Z-score calculation for days right after the shift — are they still detectable as anomalous once the "after" period dominates the average?
4. **Diagnostic question — distributional shape:** P003 has a lumpy, mostly-zero sales pattern (many true zero days by design, not anomalies). Plot the distribution of `sales` for P003. Does it look anything like a normal/symmetric distribution? What happens when you compute Z-scores on a skewed distribution like this — do the `|z| > 3` flags for P003 make intuitive sense, or does the method break down?

5. **Diagnostic question — threshold sensitivity:** Rerun with thresholds `|z| > 2` and `|z| > 4` instead of 3. How much does the flagged count change for each product? What does this tell you about how sensitive Z-score results are to an arbitrary threshold choice?

Exercise 2 — Modified Z-score (MAD-based) Detection

Goal: Learn a robust alternative to Z-score, and prove to yourself why it's more robust rather than just taking that claim on faith.

1. For each product, compute the median and the **Median Absolute Deviation (MAD)**: `MAD = median(|x - median(x)|)`.

2. Compute the modified Z-score for every row:
   `M = 0.6745 * (x - median) / MAD`.
   Flag `|M| > 3.5` (the commonly used threshold for this method).

3. **Robustness test :** For P001 or P002, take the single most extreme value in the series (from your Exercise 1 results) and:

   - Compute mean and standard deviation **with** that point included, and **without** it (remove that one row and recompute).
   - Compute median and MAD **with** and **without** that same point.
   - Report all four before/after pairs in a table. Which pair of statistics (mean/std or median/MAD) shifts more when that single extreme point is removed? This is the concrete evidence for why MAD is called "robust."

4. **Comparison question:** Build a table showing, per product, how many rows were flagged by Z-score (Exercise 1, `|z|>3`) vs. modified Z-score (`|M|>3.5`). Pick two or three specific rows where the methods *disagree* (flagged by one, not the other) and explain, using the actual numbers, why each method reached the answer it did.

5. **Reflection question:** Modified Z-score still assumes you're looking for outliers relative to *one* global center (the median) for the whole two-year period. Name one anomaly type in this dataset that you'd expect **neither** Z-score nor modified Z-score to catch reliably, and explain why a "distance from one fixed center"

approach is the wrong tool for it. (You don't need to detect it yet — just reason about it. You'll revisit this in Exercise 3.)

Exercise 3 — KNN Distance-Based Detection

Goal: Move from a single global statistic to a method that judges each point by its *neighbors* — and that can use more than one feature at once.

Part A — Univariate KNN (comparable baseline)

1. For each product, using only the `sales` column reshaped as a single feature, fit a `NearestNeighbors` model (scikit-learn) with `k=5`.
2. For every row, compute its **average distance to its k nearest neighbors** in `sales`-space. Flag the rows with the highest average distance (e.g., top 2–3% per product, or a distance threshold you justify).
3. Compare this univariate-KNN flag list to your Z-score and modified-Z-score lists from Exercises 1–2. On sales-only data, do you expect KNN to behave similarly to modified Z-score? Check whether it does, and explain any differences you find.

Part B — Multivariate KNN (the actual point of using KNN)

1. Engineer a small feature set per row, per product:
   - `sales` (the raw value)
   - `sales_lag_1` (previous day's sales)
   - `sales_lag_7` (same day, previous week)
   - `rolling_mean_7` (trailing 7-day average, using only past data)
   - `day_of_week`
2. Standardize/scale the features (e.g., `StandardScaler`) — this matters for KNN since it's distance-based and unscaled features with different ranges will dominate the distance calculation. Explain in one or two sentences what would go wrong if you skipped scaling.
3. Fit `NearestNeighbors` with `k=5` on this multivariate feature space, per product. Compute each row's average distance to its k nearest neighbors and flag the highest-distance rows (same top-2–3% approach as Part A, or a threshold you justify).

4. **Key comparison:** Revisit your answer to Exercise 2's reflection question (the anomaly type you predicted neither Z-score nor modified Z-score would catch well). Does multivariate KNN catch it? Explain *why*, referencing which of your engineered features gave KNN the information Z-score/modified-Z-score never had access to.
5. **Discussion question:** KNN with lag/rolling features can catch **contextual** anomalies (a value that's normal in general but wrong *for that moment in the sequence*) in a way that plain Z-score/modified Z-score — which only look at the raw value against one global center — structurally cannot. Explain this distinction in your own words, using one concrete example row from the dataset as evidence.
6. **Practical trade-off question:** For each of the three methods (Z-score, modified Z-score, multivariate KNN), state: (a) how many parameters/choices you had to make to run it, (b) whether you can explain *why* a specific point was flagged in plain language to a non-technical stakeholder, and (c) how the method would need to change if a fifth product were added to the dataset tomorrow. Which method would you put into a live monitoring pipeline, and which would you keep for offline investigation only? Justify your choice.

Exercise 4 — Scoring All Three Methods Against Ground Truth

Goal: Check your work.

1. Now open `anomaly_answer_key.csv`. It lists every anomaly actually injected into the dataset (type, location, and the original vs. injected values).
2. For each of your three detectors (Z-score, modified Z-score, multivariate KNN), compute:
   - **Precision**: of the rows you flagged, what fraction were true anomalies?
   - **Recall**: of the true anomalies, what fraction did you catch?
3. Break precision/recall down **by anomaly type**: `point_spike`, `stockout_zero_run`, `level_shift_start`, `data_entry_error`, `missing_record`, `duplicate_record`, `gradual_drift_start`. Present this as one table with methods as rows and anomaly types as columns (recall per cell).
4. **Note before you start:** `missing_record` and `duplicate_record` are structural anomalies, not value anomalies — a row that doesn't exist can't get a Z-score, and a duplicated row won't necessarily look numerically strange on its own. None of the three

methods in this exercise set are built to catch these two types by construction. Confirm this is what you observe, and explain in one sentence why it's an expected result rather than a bug in your code.

5. **Final question:** Across `point_spike`, `stockout_zero_run`, `level_shift_start`, `data_entry_error`, and `gradual_drift_start` — rank the three methods from best to worst at recall, and separately from best to worst at precision. Do the same method rank best on both? If not, explain the trade-off in your own words.